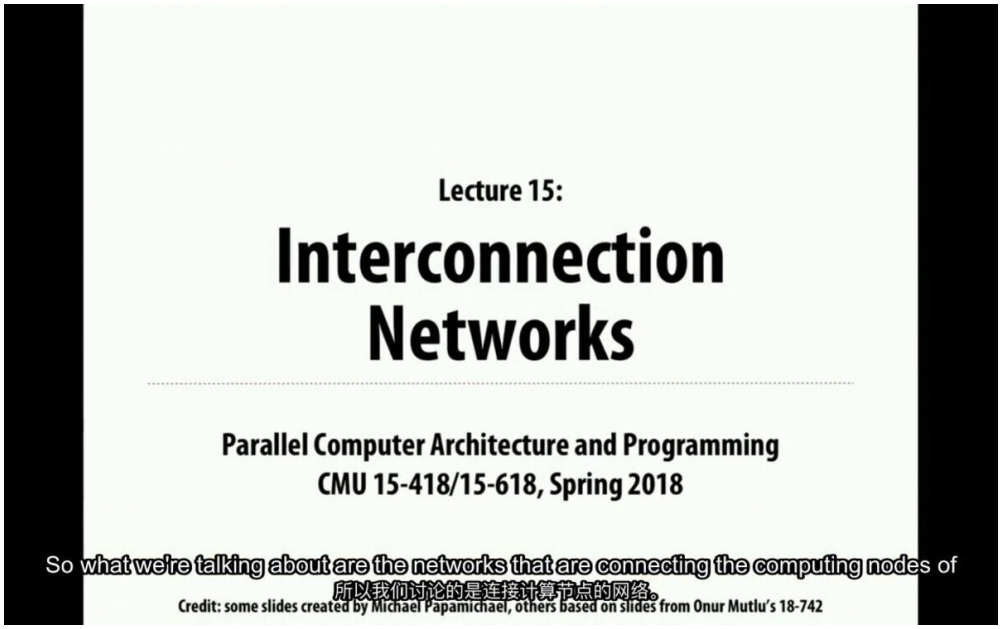
课程笔记

互连网络：拓扑、对分带宽与可扩展性

CMU 15-418/618 并行计算机架构与编程 · Lecture 19（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 14 分 38 秒
视频链接：`https://www.bilibili.com/video/BV1Nw41197ZZ/`

目录

1 互连网络：一个在小规模下看不见的问题 2

1.1 此前我们一直假设有个总线 . . . 2
1.2 总线的物理构造 . . . 3
1.3 术语 . . . 3
1.4 本章小结 . . . 4

2 为什么互连设计越来越重要 4

2.1 三个权衡维度 . . . 4
2.2 真实机器的规模 . . . 5
2.3 本章小结 . . . 6

3 拓扑、路由、缓冲与流量控制 6

3.1 四个设计问题 . . . 6
3.2 拓扑的两个度量 . . . 6
3.3 对分带宽 . . . 7
3.4 阻塞与非阻塞 . . . 8
3.5 负载与延迟的关系 . . . 9
3.6 本章小结 . . . 10

4 拓扑巡礼：从交叉开关到环 10

4.1 交叉开关 . . . 10
4.2 本章小结 . . . 11

5 小规模方案：交叉开关与环 11

5.1 交叉开关的面积代价 . . . 11
5.2 环 . . . 12
5.3 Intel 的环 . . . 12
5.4 本章小结 . . . 13

6 网格、环面与折叠环面 13

6.1 mesh . . . 13
6.2 Xeon Phi 的 mesh . . . 14
6.3 环面 . . . 14
6.4 折叠环面 . . . 15
6.5 本章小结 . . . 15

7 树、胖树与超立方体 15

7.1 树与它的路由 . . . 15
7.2 树根的瓶颈与胖树 . . . 16
7.3 等宽胖树 . . . 17
7.4 超立方体 . . . 17
7.5 omega 网络 . . . 18
7.6 本章小结 . . . 18

8 总结与延伸 **19**

1　互连网络：一个在小规模下看不见的问题

这一讲讨论连接高性能计算系统中各计算节点的网络。讲者开场就做了一个区分：你在网络课上学的互联网与 TCP 都是很有意思的主题，但这些网络很不一样。

它们的设计目标是紧密局部、极高性能；另一方面，它们不必像互联网那样担心故障与断连。这类网络要连接的规模可以很大——超算中心可能有 10000 个节点需要互相连通。

1.1　此前我们一直假设有个总线

讲者回顾说，前面讲缓存协议时，我们只是假设缓存之间能以某种方式通信，通常就直接假设它是一条总线。

需要记住的是：这些连接点被称为节点，但实际通常是缓存控制器、内存控制器以及其他系统部件，它们充当处理器的代理。今天统称为节点。

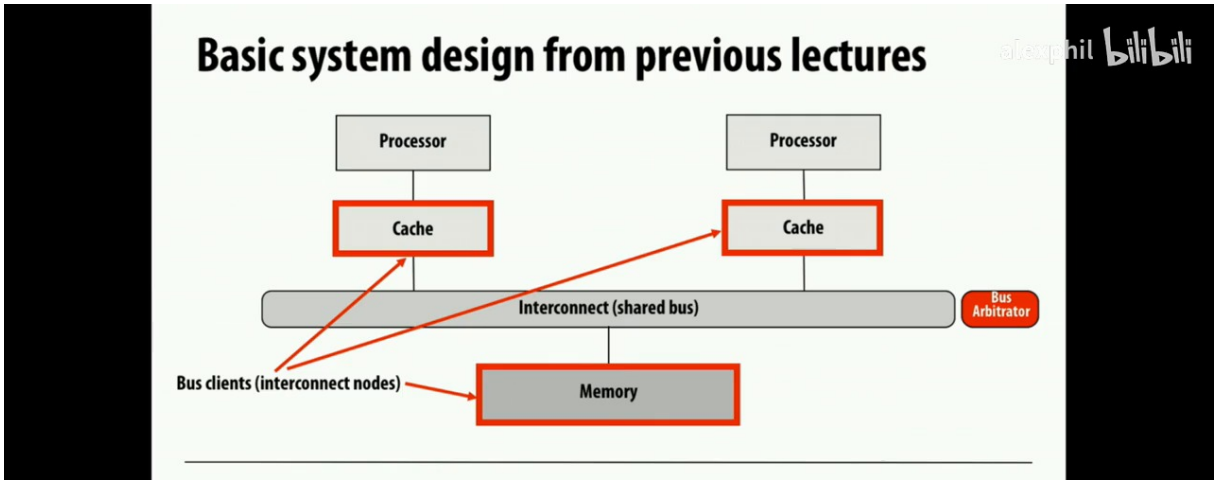


图 1: 前几讲的基本系统设计：处理器、缓存与互连的位置[1]

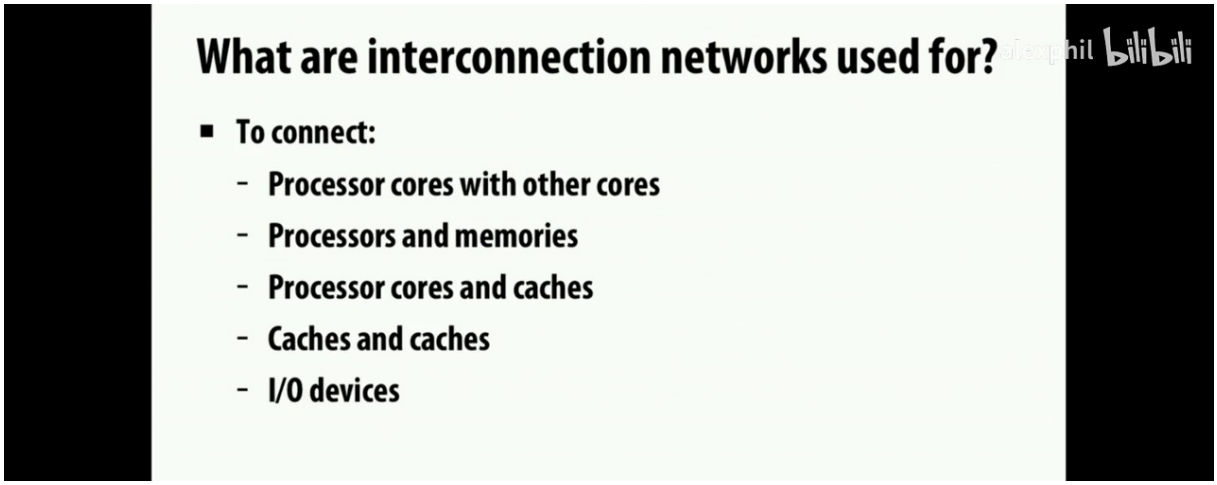


图 2: 互连网络用来做什么：连接核与核、核与内存、核与 IO[2]

[1] 视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。

[2] 视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。

1.2 总线的物理构造

总线是这件事最简单的形式，实际上就是一组导线。一组用于发出请求——把地址放上去；另一组用于返回响应，也就是读操作取回的数据。

讲者强调一个关键设计：这两组通常做成物理上分离的总线，以避免死锁——否则一堆请求可能堵住通道，让响应无法返回，系统就死锁了。

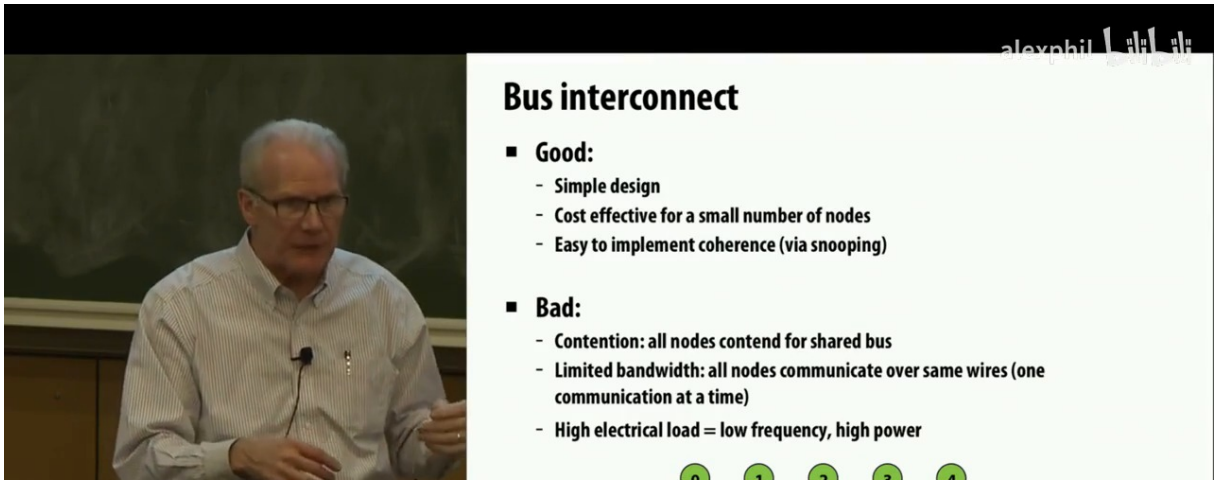


图 3: 总线互连：设计简单、对小规模节点成本有效[3]

请求与响应分离是为了避免循环等待

若请求与响应共用一条通道，当大量请求占满通道时，响应无法返回；而发出请求的一方在等到响应之前不会释放通道，于是形成循环等待。物理分离两条总线打断了这个环。这是互连设计中”用资源换正确性”的一个典型例子。

1.3 术语

讲者给出三个基本术语。网络节点是连接到路由器或交换机的端点，可以是处理器核、缓存控制器、内存控制器或 IO 设备。

链路是网络中的点对点连接，物理上通常是一束导线加两端能收发消息的电路。

交换机是一个能把消息沿不同路径导向目的地的模块。讲者说交换机内部的逻辑正是今天要看的一部分，它定义了路由元素。

[3]视频画面时间区间：00:20:30–00:20:45。若为裁剪图，时间区间仍对应原视频画面。

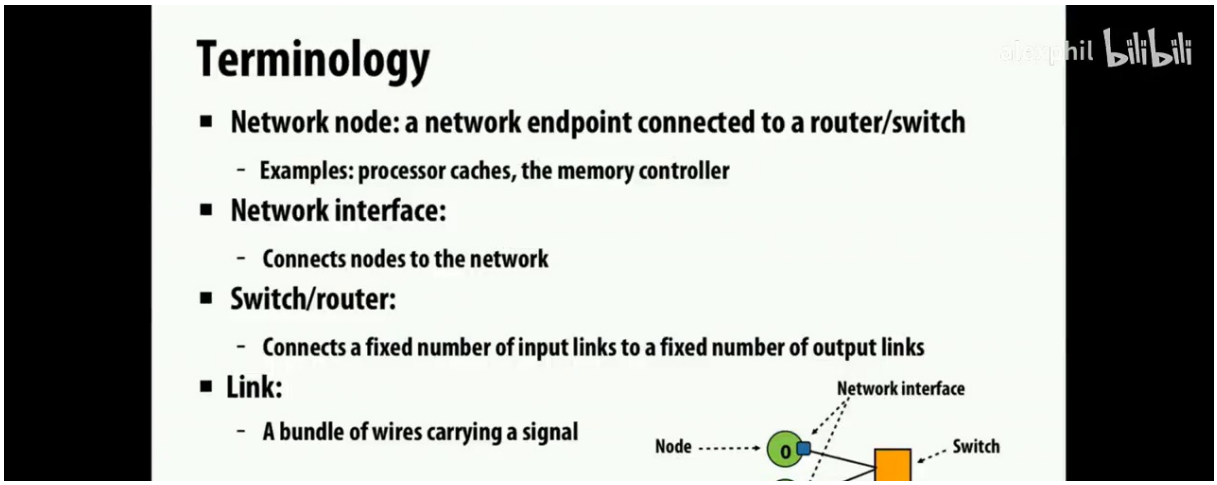


图 4: 术语：网络节点、链路与交换机[4]

1.4 本章小结

互连网络连接 HPC 系统的各节点，其目标与互联网不同：紧密局部、极高性能，但可以较少考虑故障。最简单的形式是总线，请求与响应需物理分离以避免死锁。三个基本术语是节点、链路与交换机。

下一章看为什么这个问题会变得重要，以及设计时需要权衡哪些维度。

2 为什么互连设计越来越重要

2.1 三个权衡维度

讲者列出使这个问题变得困难的几个方面。

第一是性能，包含延迟与带宽两方面。

第二是能耗，尤其是片上网络，能耗是主要关切。

第三是可扩展性。

可扩展性是一条产品约束而非技术偏好

若每个规模都需要定制网络，研发与验证成本会随规模种类线性增长。因此厂商倾向选择可递归构造的拓扑（环、mesh、胖树），即使它在极端规模下不是最优。这解释了为什么很多商用互连选择渐近指标并不最好的方案。

讲者说很少有人只想造一台系统：公司不会只做一千个处理器的机器，它们希望能卖 256、512、1024 等各种规模，并且能靠增加节点来扩展，而不必为每个规模定制一整套网络。

[4]视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

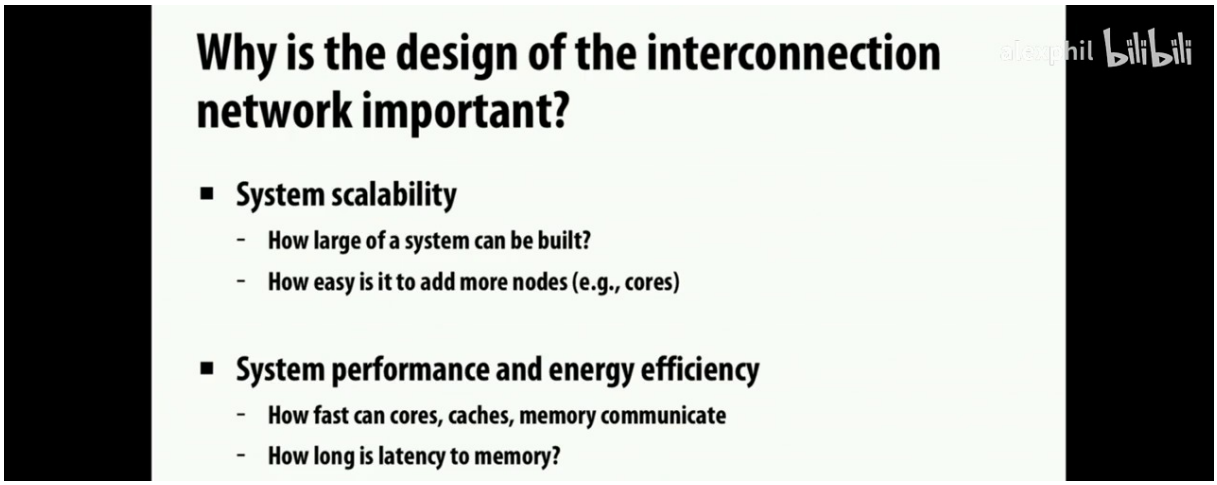


图 5: 为什么互连网络的设计很重要：系统可扩展性与成本[5]

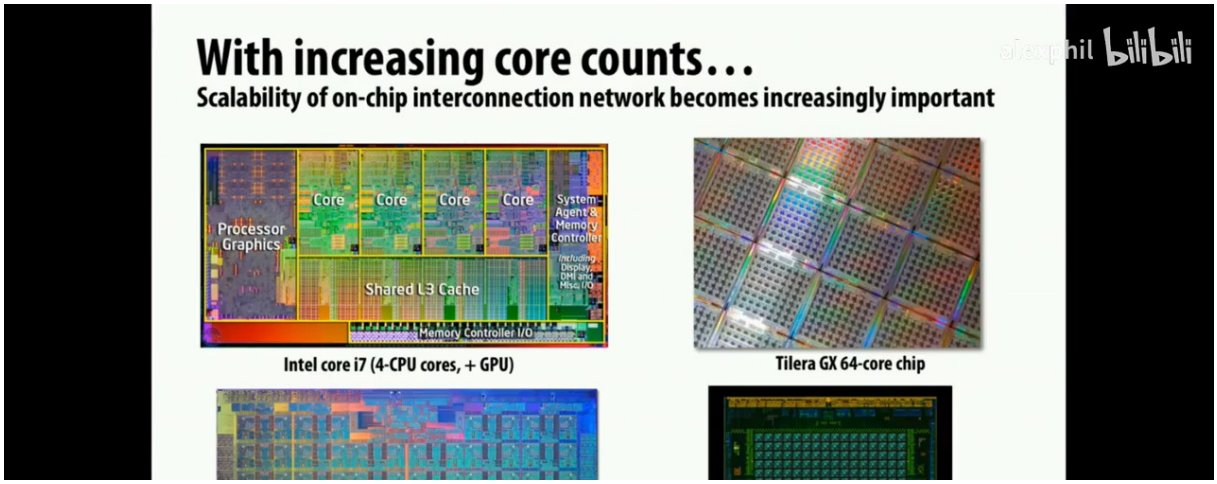


图 6: 核心数增长使片上互连的可扩展性日益重要[6]

物理上还有一个约束：在单芯片上，你可以负担更多导线、更粗的连接；跨芯片就做不到。因此系统设计层次越往上，方案差别越大。

2.2 真实机器的规模

讲者给出具体数字。课程用的多核芯片通常有六到八个核，规模相当小，它们通常用简单的环形网络。

Xeon Phi 则大得多：他们有的是 61 核机器，较新一代是 72 核。

规模决定拓扑选择

六到八核用环就够了，因为环的延迟随节点数线性增长，在个位数节点时仍可接受。一旦到几十上百个节点，环的直径就成了瓶颈，于是需要 mesh、树或胖树。拓扑选择不是偏好问题，而是由规模与流量模式共同决定的。

[5] 视频画面时间区间：00:04:00–00:04:15。若为裁剪图，时间区间仍对应原视频画面。

[6] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

2.3 本章小结

互连设计需同时权衡性能（延迟与带宽）、能耗与可扩展性。厂商希望同一套设计覆盖 256 到 1024 等多种规模。物理约束使片上与跨芯片方案差别很大。真实机器中，六到八核用环，61 到 72 核的 Xeon Phi 则复杂得多。

要比较不同方案，需要一套描述与度量工具。

3 拓扑、路由、缓冲与流量控制

3.1 四个设计问题

讲者把设计问题拆成四类，这个划分贯穿全讲。

拓扑：网络的整体图结构，也就是交换机与链路如何连接节点。

路由：要把消息从 A 送到 B，选哪条路径，算法是什么。

缓冲：实际发送一条消息意味着什么？一个极端是必须预留整条通路，另一个极端是逐段搬运消息的一部分。

流量控制：如何管理拥塞，如何应对同一时刻大量消息都要通过的情形。

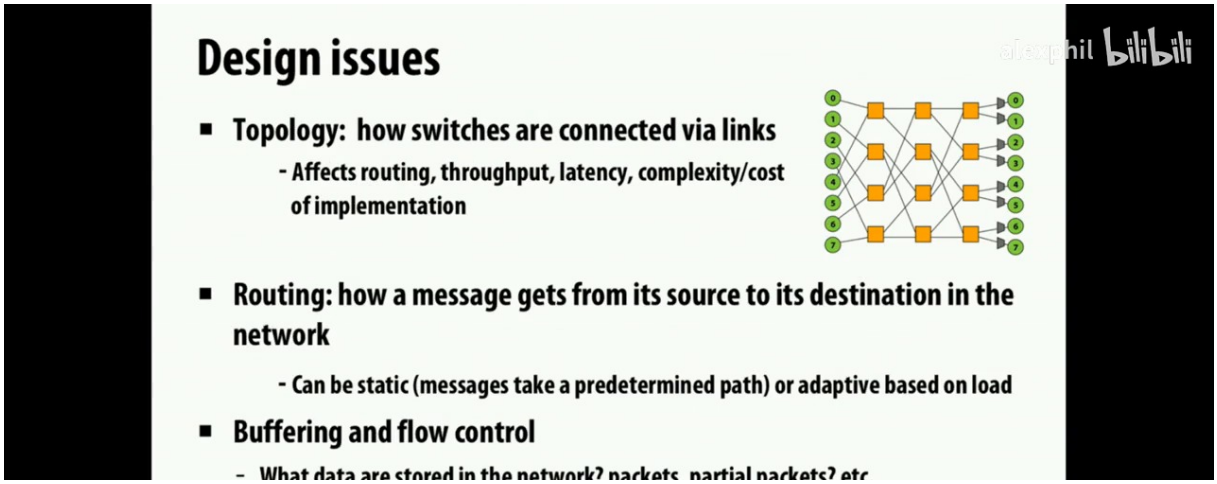


图 7: 设计问题：拓扑影响路由与吞吐，缓冲与流量控制处理拥塞[7]

3.2 拓扑的两个度量

直径与平均距离要分开看

直径是最坏情况，用于界定延迟上界；平均距离决定典型性能。对 mesh 两者接近，因此直径能代表典型行为；但对某些不规则拓扑，最坏路径可能极少出现，此时只看直径会严重低估实际性能。评估时应两者并看。

路由距离：一条路径上经过的链路数，也就是跳数。

直径：图中最长的那条最短路径。讲者用 mesh 举例：$n \times n$ 的 mesh，直径是 $2n-2$。

$$D_{\text{mesh}} = 2n - 2, \qquad D_{\text{ring}} = \frac{N}{2}, \qquad D_{\text{hypercube}} = \log_2 N$$

[7] 视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。

- D_{mesh}：$n \times n$ 网格的最坏情况跳数，从一角到对角；
- D_{ring}：N 个节点绕环一周的最远距离；
- $D_{\text{hypercube}}$：N 个节点的超立方体只需逐位翻转，因此与位数同阶。

边缘效应使 mesh 的节点不等价

mesh 中心的节点有四条出边，边缘只有三条，角落只有两条。因此位于不同位置的节点其延迟与带宽并不相同。这会给负载均衡带来额外难度：把任务均匀分配到各节点，并不意味着通信代价也均匀。环面的边界回绕正是为消除这种不等价而设计的。

他还指出 mesh 的平均情况与最坏情况差别不大。

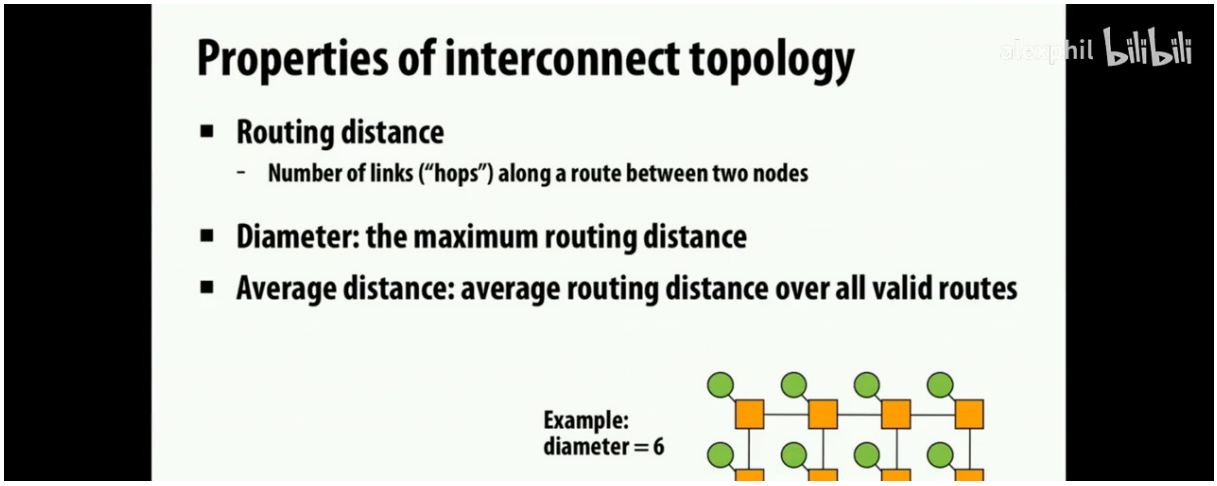


图 8: 互连拓扑的性质：路由距离（跳数）与直径[8]

3.3 对分带宽

这是讲者认为最有用的性能指标：把网络切成两半，数一数跨越切口的链路数。

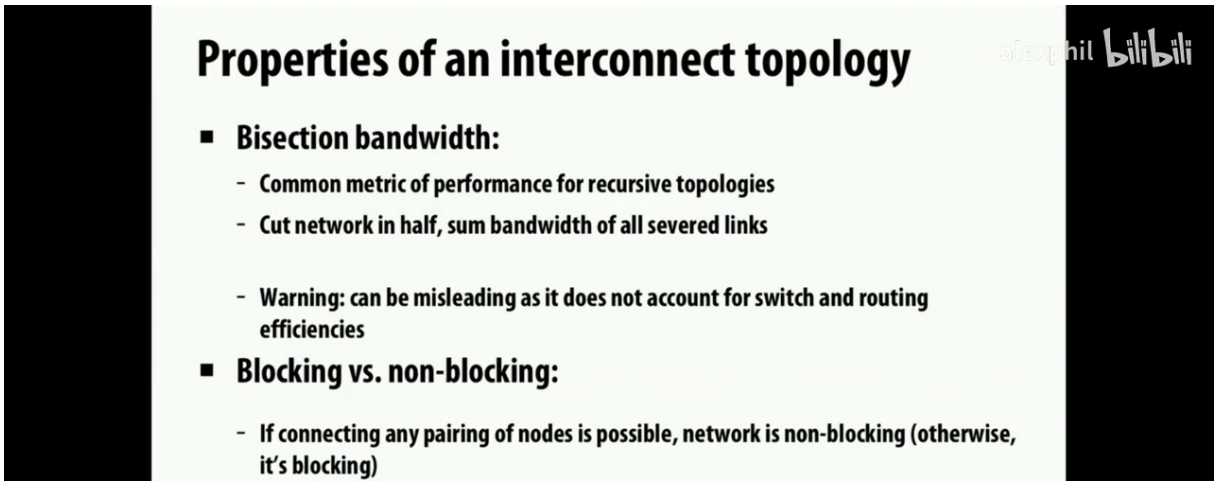


图 9: 对分带宽：衡量网络连通度的通用性能指标[9]

[8] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

- 对 $n \times n$ 的 mesh，切成两半会切过约 n 条线。节点数是 n^2，因此对分带宽是节点数的平方根——讲者的评价是” 好，但不是很好”。
- 对简单的环或线性链，对分带宽是 1：切断任意一条线网络就断了。

写成通式：

$$B_{\text{mesh}}(N) = \sqrt{N}, \qquad B_{\text{ring}}(N) = 1, \qquad B_{\text{hypercube}}(N) = N$$

- N：网络节点总数；
- mesh 的对分带宽随 $\sqrt{N}$ 增长，环保持常数，超立方体随 N 线性增长；
- 三者的差距随规模扩大而急剧拉开。

为什么这重要？讲者给出具体场景：如果应用完全没有局部性，比如你正在做的作业，那么大量流量都要穿过系统，低对分带宽就成了瓶颈。

他做了个估算：若是完全随机的置换，每个人都去取数据，在中线处看过去，大约四分之一留在这一半、四分之一留在另一半、四分之一要从这半边到那半边，还有四分之一反向。

这可以写成跨越中线的流量比例：

$$f_{\text{cross}} = \frac{1}{4} = 0.25$$

- f_{cross}：随机置换下需要跨越中线的流量占总流量的比例；
- 每个节点对均匀分布时，四分之一的对子跨越中线；
- 若对分带宽为 B，则需承载 $0.25 \cdot N$ 量级的跨线流量。

这正是低对分带宽在随机访问下成为瓶颈的定量原因。

对分带宽是” 最坏情况” 的度量

它衡量的是网络在被切成两半时的最大承载能力，因此对无局部性的随机访问模式最敏感。若应用的通信有强局部性，实际瓶颈可能不在对分带宽而在局部链路。因此判断一个拓扑是否合适，必须同时看访问模式与这个指标。

3.4 阻塞与非阻塞

讲者用例子讨论这个概念：给定一个网络，同时让所有节点对之间都通信，是否都能无冲突地完成？能则是非阻塞，不能则是阻塞。

[9] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

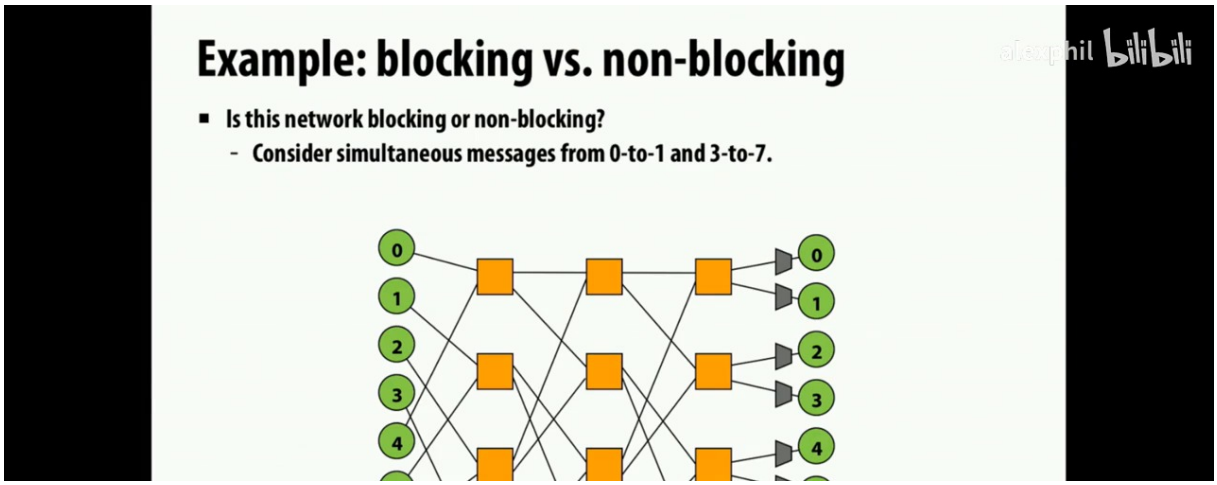


图 10: 例子：给定网络在同时通信时是否阻塞[10]

3.5 负载与延迟的关系

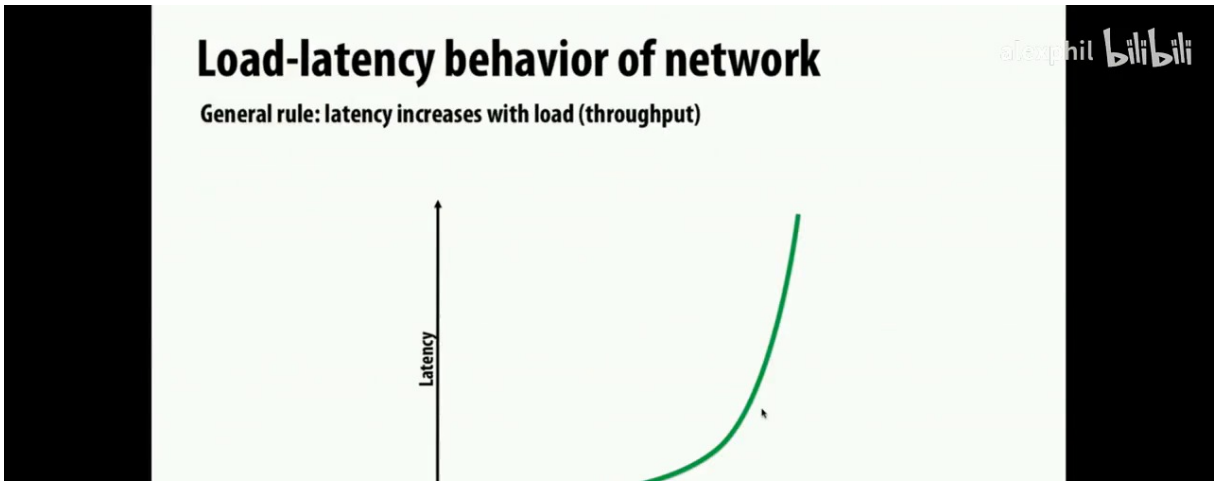


图 11: 网络的负载-延迟行为：延迟随负载上升而增加[11]

讲者给出一条通用规律：延迟随负载（吞吐）增加而增加。这是所有互连网络共有的行为——轻载时延迟接近理论最小值，接近饱和时延迟急剧上升。

网络性能必须用”负载-延迟曲线”描述而非单一数字

只报一个延迟值或一个带宽值都不足以刻画网络。真正决定应用性能的是在特定负载下的延迟，而负载又取决于应用的通信模式。因此评价网络要看曲线形状：拐点在哪里，饱和点在哪里。

[10] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。
[11] 视频画面时间区间：00:17:00–00:17:15。若为裁剪图，时间区间仍对应原视频画面。

3.6 本章小结

设计问题分四类：拓扑、路由、缓冲、流量控制。拓扑用路由距离与直径刻画，用对分带宽衡量连通度：mesh 为节点数的平方根，环为 1，超立方体为节点数。无局部性负载会让低对分带宽成为瓶颈。阻塞与非阻塞描述并发通信能力，而延迟随负载上升是通用规律。

有了这套度量，就可以逐个比较具体拓扑了。

4 拓扑巡礼：从交叉开关到环

4.1 交叉开关

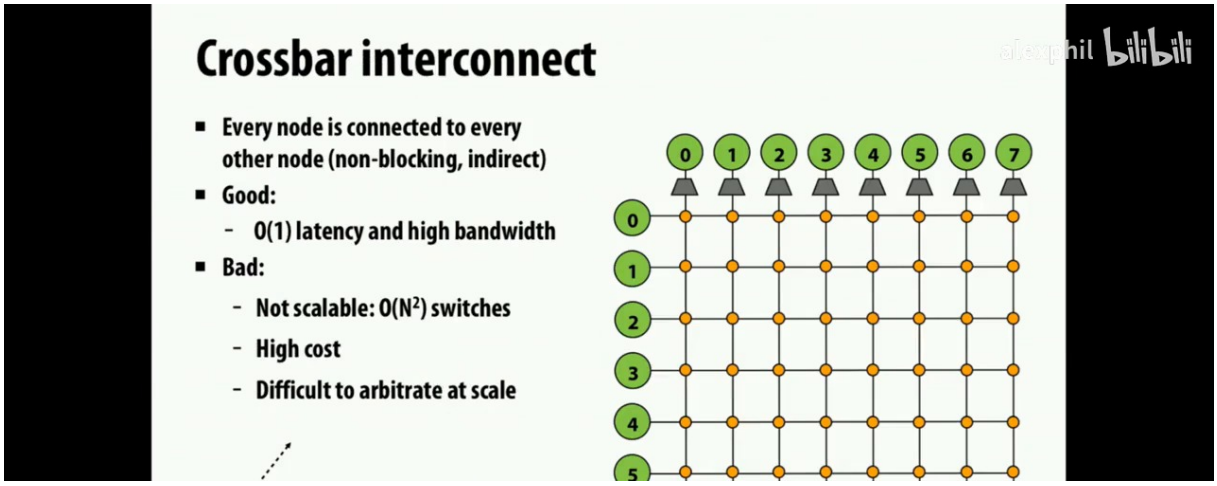


图 12: 交叉开关：每个节点与其他所有节点直连，非阻塞[12]

交叉开关是非阻塞的间接网络：每个节点都与其他所有节点相连。本质上它是一张完整图。

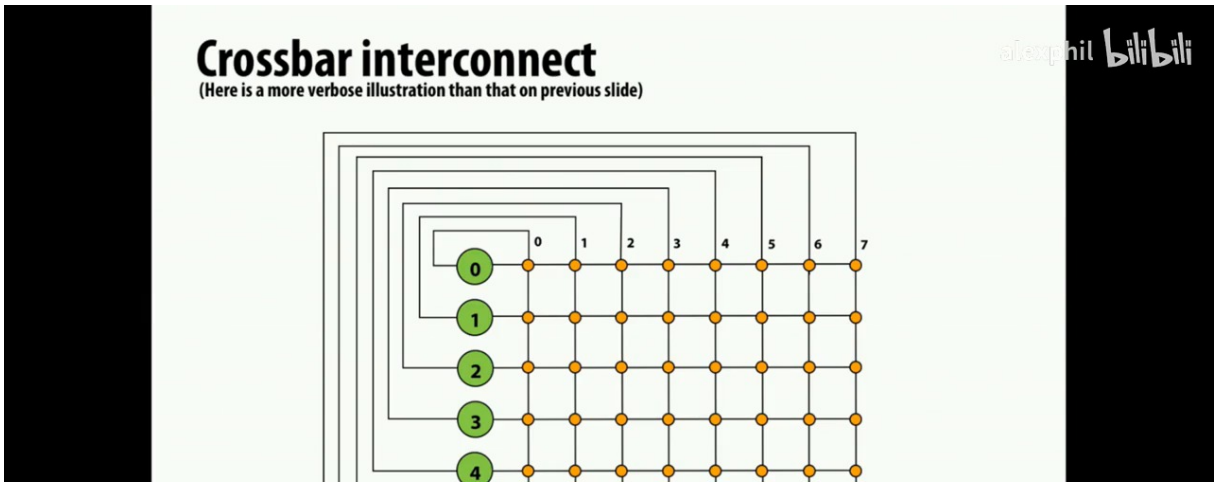


图 13: 交叉开关的更详细图示[13]

讲者澄清了一个容易误读的图示：那张图看起来像 mesh，需要逐跳再跳一下，但它的本意是完整图——每个节点到每个其他节点都有直连，不受限制。

[12] 视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。

[13] 视频画面时间区间：00:25:30–00:25:45。若为裁剪图，时间区间仍对应原视频画面。

4.2 本章小结

拓扑度量用路由距离与直径刻画延迟，用对分带宽度量连通度：mesh 为 $\sqrt{N}$、环为 1、超立方体为 N。三者差距随规模扩大而急剧拉开。阻塞与非阻塞描述并发通信能力，而延迟随负载上升是通用规律。无局部性的随机访问会让低对分带宽成为瓶颈——随机置换下约四分之一流量需跨越中线。

有了这套度量，就可以逐个比较具体拓扑。

5 小规模方案：交叉开关与环

5.1 交叉开关的面积代价

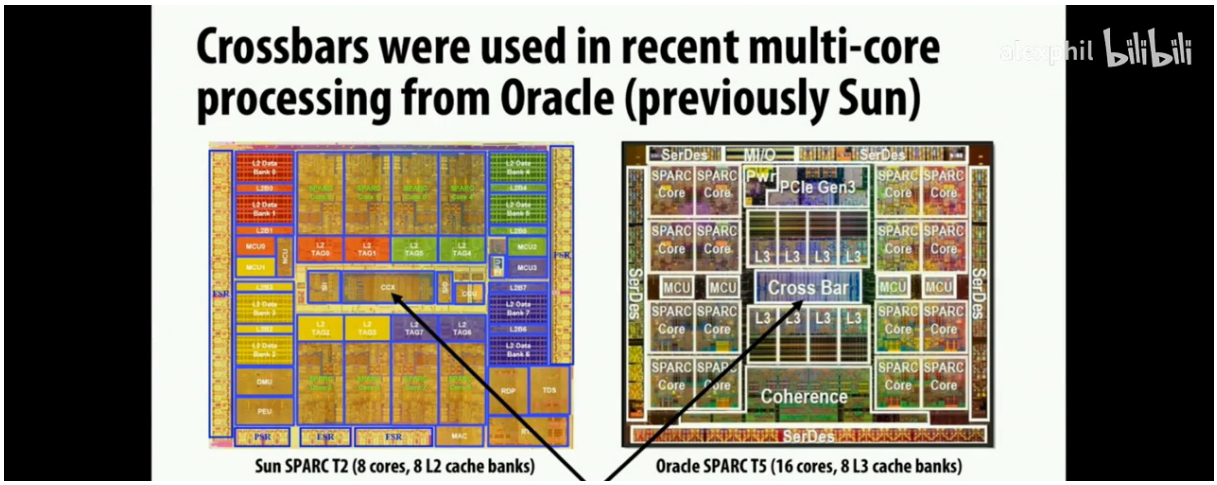


图 14: 交叉开关在 Oracle（原 Sun）多核处理器中的实际使用[14]

讲者展示了 Oracle（原 Sun）的一颗多核芯片。芯片内有一个交叉开关，物理上它占的面积大约相当于一个核——图中竖着的矩形是核。因此这是相当可观的一块面积。

但在芯片上，你确实可以造出交叉开关所需的连通度。这说明了交叉开关的适用边界：它提供最好的连通性，代价是面积随节点数平方增长。

$$\text{面积}_{\text{crossbar}} \propto N^2$$

- N：交叉开关连接的节点数；
- 每条连线交叉处都需要开关资源，因此面积按平方增长；
- 这决定了交叉开关只适用于小规模（如单芯片内的几个核）。

[14] 视频画面时间区间：00:24:00–00:24:15。若为裁剪图，时间区间仍对应原视频画面。

5.2 环

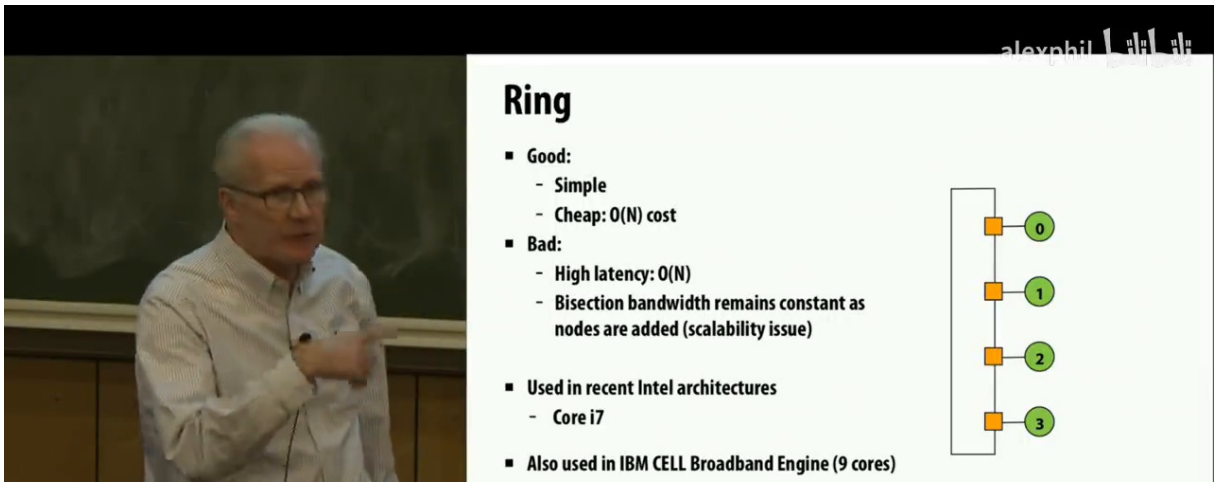


图 15: 环形互连：简单、成本 $O(N)$，但延迟 $O(N)$、对分带宽恒定[15]

环处在另一个极端：消息绕环流动。优点是极易扩展——加倍规模只需加倍环节点。成本是 $O(N)$。

缺点是延迟 $O(N)$，且对分带宽保持不变（因为切断两处就断网）。

讲者说环确实被实际使用，但通常只在四到八个元素的规模上。

5.3 Intel 的环

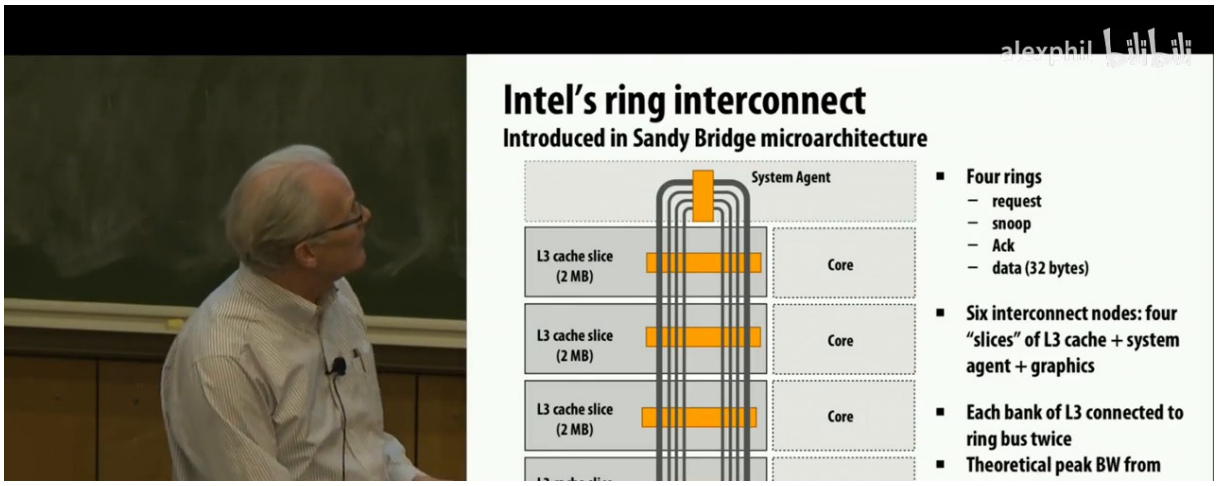


图 16: Intel 的环形互连：Sandy Bridge 引入，四条环连接缓存控制器[16]

讲者给出一个具体例子：Intel 的环形互连，从 Sandy Bridge 微架构引入。图中可以看到四条环，以及 System Agent。

他补充说最新一代叫 Skylake，用的是 mesh 网络，而之前各代都只用环连接缓存控制器。

他还指出课程集群里的 Xeon Phi 是环连接的，这也是它们性能不佳的部分原因。

[15] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

[16] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

Intel 从环迁移到 mesh 说明了什么

环在四到八个核时够用，但当核数增长到几十个，环的 $O(N)$ 延迟与恒定的对分带宽成为瓶颈。Skylake 改用 mesh 正是在这个压力下的必然选择：mesh 的对分带宽是节点数的平方根，延迟是 $O(\sqrt{N})$，在中等规模上明显优于环。

5.4 本章小结

交叉开关提供完整图连通性，面积按平方增长，只适合单芯片内小规模。环成本 $O(N)$、延迟 $O(N)$、对分带宽恒定，适合四到八个元素，Intel 曾用它连接缓存控制器，Skylake 才转向 mesh。Xeon Phi 用环也是其性能不佳的部分原因。

6 网格、环面与折叠环面

6.1 mesh

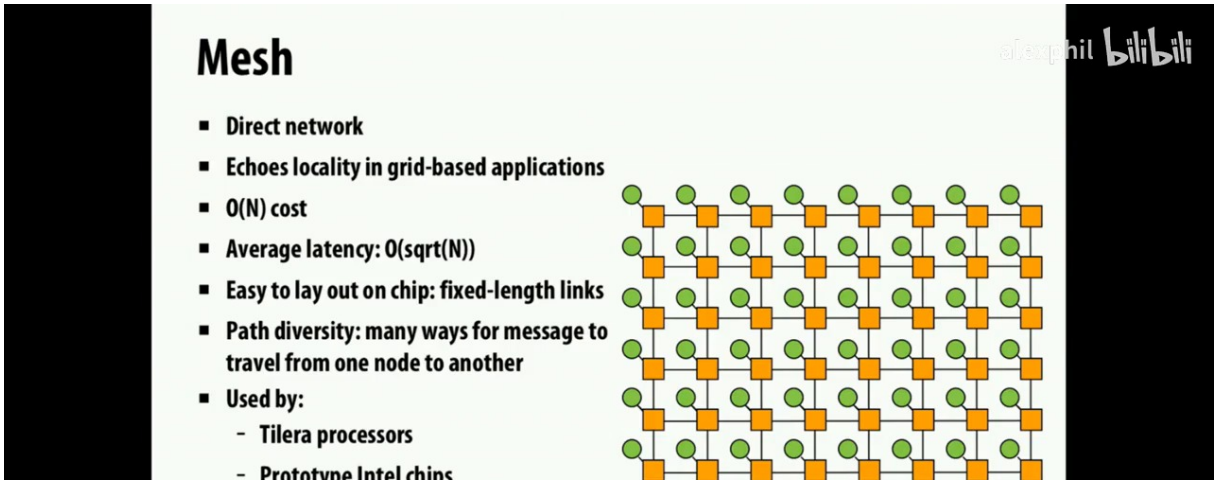


图 17: mesh：直接网络，呼应基于网格的应用的局部性，成本 $O(N)$[17]

mesh 是直接网络——交换机与节点是同一件事，交换逻辑内建在节点里。

拓扑与应用的局部性应当匹配

网格类应用（如二维分块的矩阵运算、格点模拟）的通信天然发生在相邻格点之间，因此 mesh 的物理布局与应用的数据布局同构，通信路径短且可预测。若把这类应用放到胖树上，相邻格点的数据可能被放到相距很远的叶子节点，反而增加跳数。

讲者说它呼应了基于网格的应用的局部性，成本是 $O(N)$，平均延迟优于环。

[17] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。

6.2 Xeon Phi 的 mesh

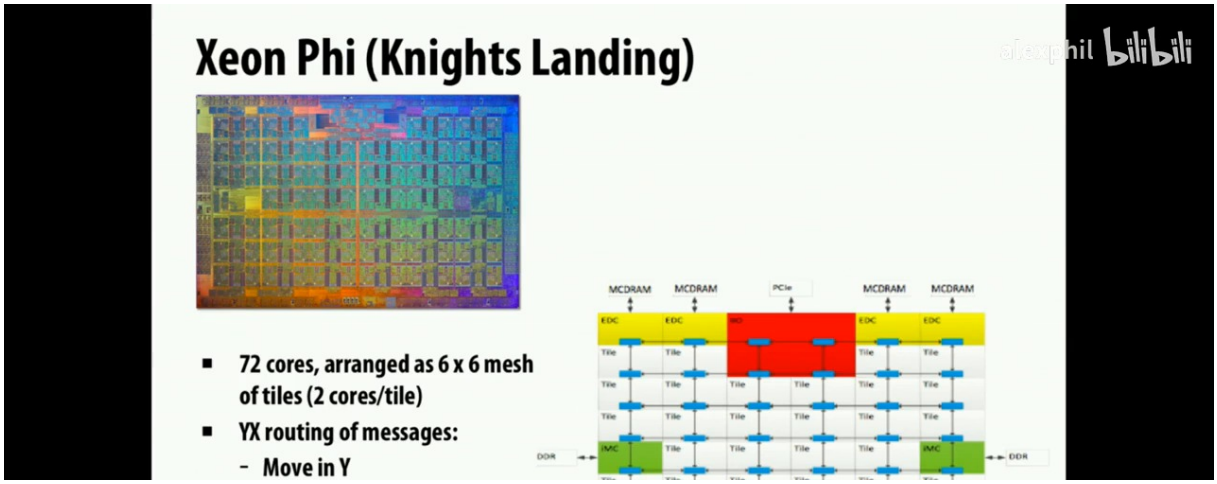


图 18: Xeon Phi（Knights Landing）：72 核排列为二维网格，含 MCDRAM[18]

讲者展示 Knights Landing 的布局：72 个核排列成二维阵列，周围是 MCDRAM（高带宽片上内存）。

6.3 环面

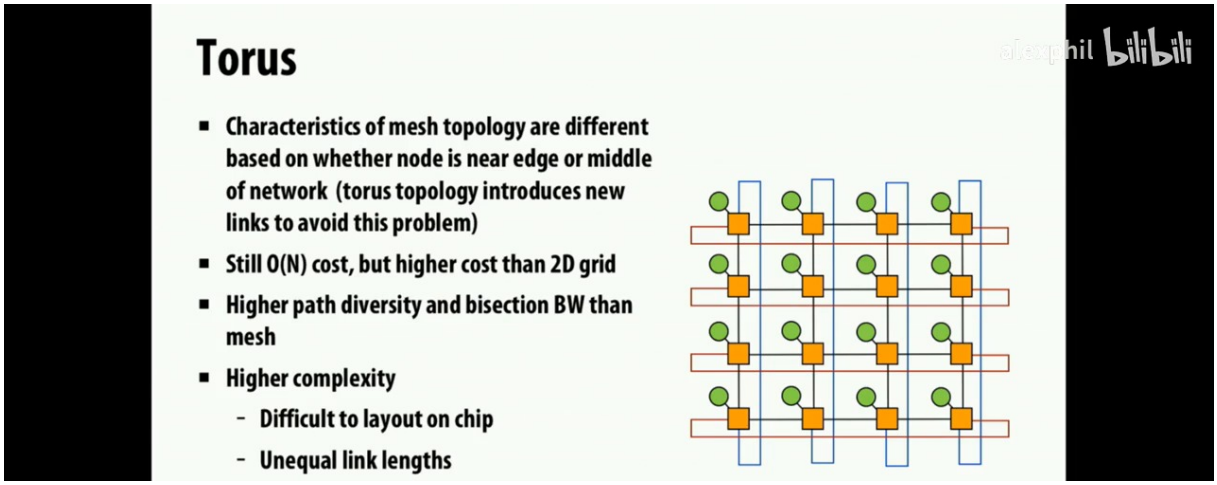


图 19: 环面：mesh 的边界回绕，使边缘节点的连通性接近中心[19]

环面就是 mesh 加上边界回绕。讲者指出 mesh 的特性会因节点位于边缘还是中间而不同，而回绕消除了这种差异。

[18] 视频画面时间区间：00:30:30–00:30:45。若为裁剪图，时间区间仍对应原视频画面。
[19] 视频画面时间区间：00:31:30–00:31:45。若为裁剪图，时间区间仍对应原视频画面。

6.4 折叠环面

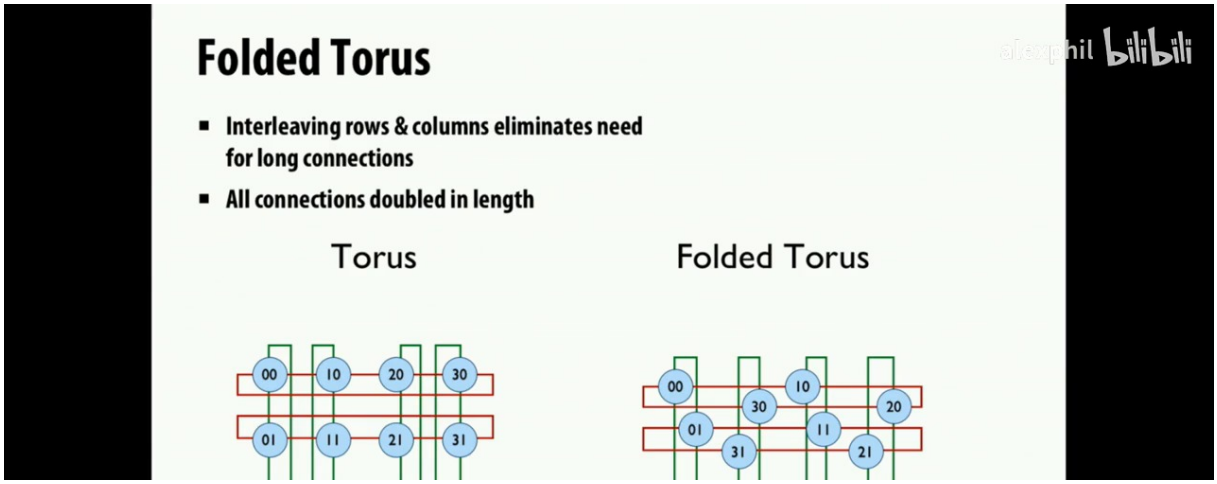


图 20: 折叠环面：交错行列以消除长连线，所有连接等长[20]

折叠环面通过交错行列来消除长连接的需要，使所有连接长度相同。这是物理实现上的优化——长导线在芯片上代价很高。

6.5 本章小结

mesh 是直接网络，交换逻辑在节点内，成本 $O(N)$，对分带宽为节点数的平方根，且呼应网格类应用的局部性。环面加边界回绕消除边缘与中心的差异，折叠环面进一步让所有连接等长以适应物理布局。

这些拓扑都依赖通信有局部性。若流量模式是全局的，树结构会更自然。

7 树、胖树与超立方体

7.1 树与它的路由

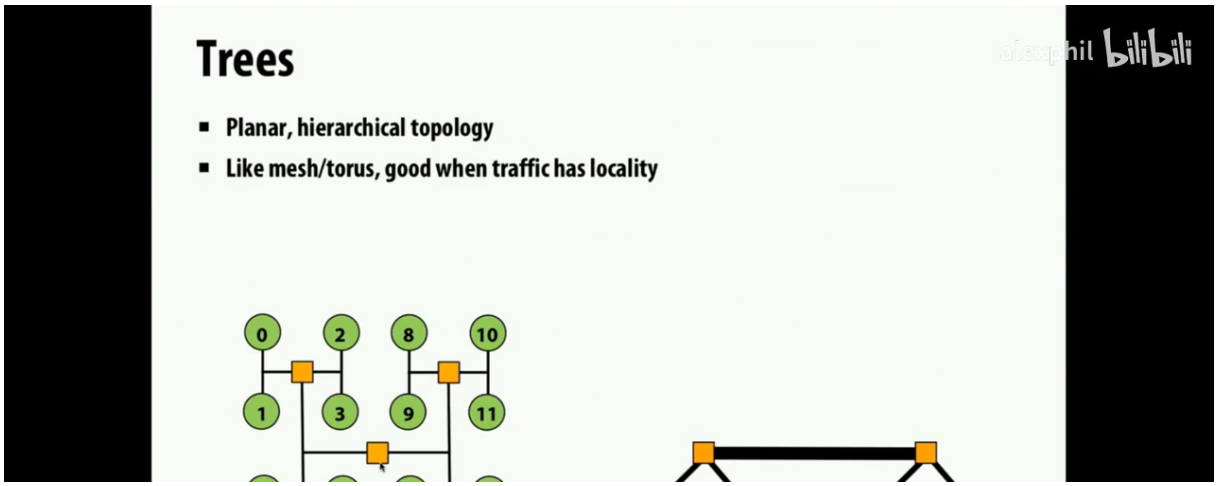


图 21: 树：平面层次化拓扑，流量有局部性时表现好[21]

[20] 视频画面时间区间：00:32:30–00:32:45。若为裁剪图，时间区间仍对应原视频画面。

树是平面层次化拓扑。和 mesh、环面一样，流量有局部性时它表现好。

树的路由极其简单，讲者用例子说明：

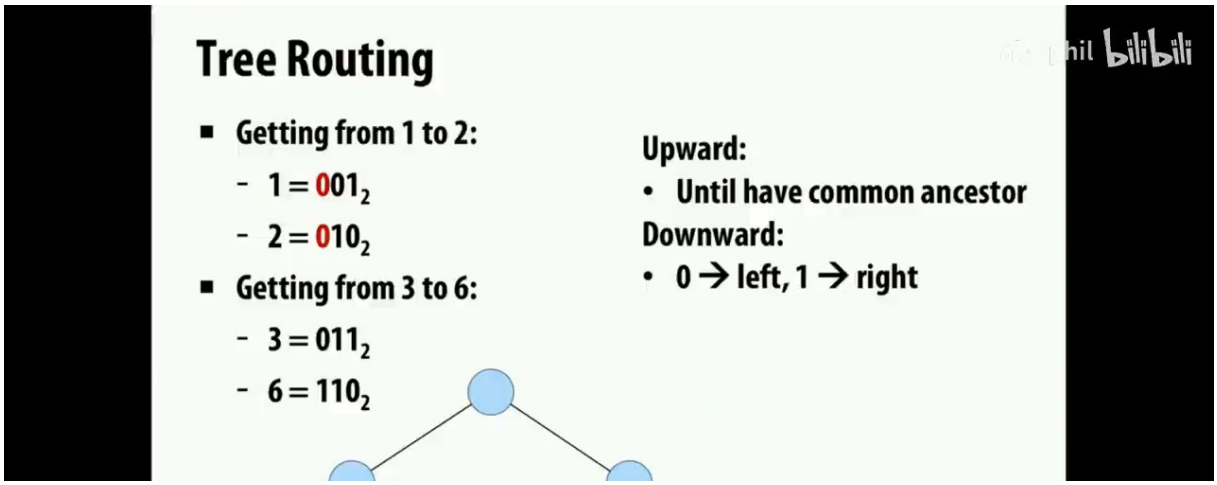


图 22: 树路由：上行到源与目的地址有公共祖先的层，再按位下行[22]

要从节点 1 到节点 2，先向上走。问题是走到哪停？基本规则是看发送方与接收方地址的位表示，上行到它们剩余低位相同的那一层。

例子里，从 1 到 2 需要上行两个位置，再往上就是通往根的共享路径，不必走。

下行时规则很简单：右是 1，左是 0，逐位进行。要到 2 就先右后左。

若要 3 到 6，没有公共祖先，就得一路走到根，然后按位下行 1、1、0。

树路由是位运算，不需要路由表

路由决策完全由源与目的地址的位决定，无需在每个交换机里维护转发表。这让树的硬件极其简单、延迟可预测。代价是树的连通性差，一旦某条上层链路成为热点就无法绕行。

7.2 树根的瓶颈与胖树

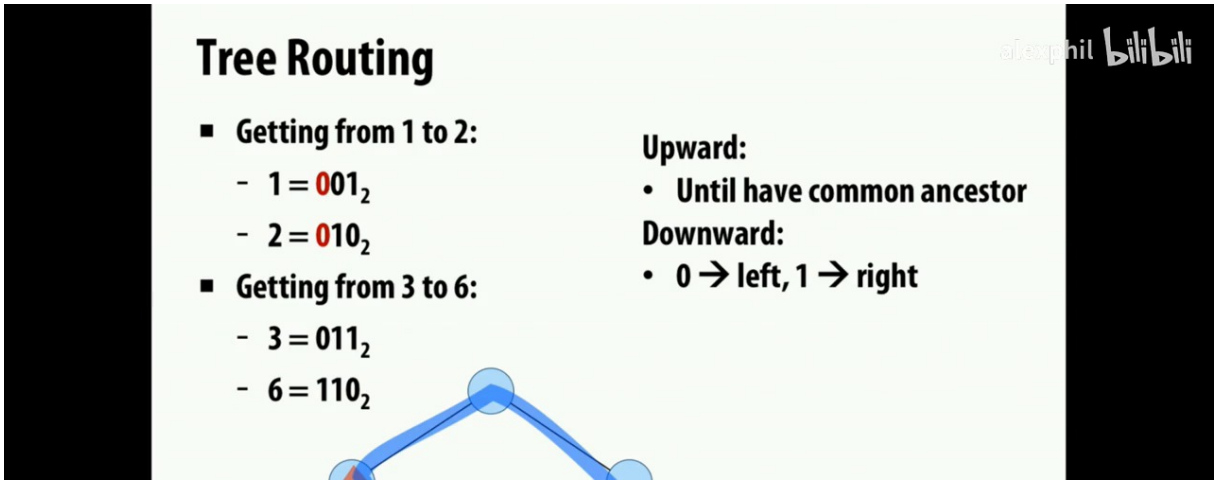


图 23: 胖树：在上层增加链路，使对分带宽不随深度受限[23]

[21] 视频画面时间区间：00:35:30–00:35:45。若为裁剪图，时间区间仍对应原视频画面。

[22] 视频画面时间区间：00:38:00–00:38:15。若为裁剪图，时间区间仍对应原视频画面。

经典树的问题是在根部有可怕的瓶颈。讲者说胖树的想法由 Charles Leiserson 提出（他在 CMU 获得博士学位，此后一直在 MIT 任教）。

他的观察正是讲者前面提到的：在树的上层，我们可以负担得起增加更多链路。这个设计里，若从中间切开，对分带宽是四条链路——和任何位置切开一样好。

胖树用”上层加宽”换取对分带宽

普通树的链路数在下层多、上层少，因此上层成为瓶颈。胖树让上层链路数成倍增加，使任意切口的对分带宽相同。代价是上层需要更多物理连线与交换机端口。这是用硬件资源换连通性的直接交易。

7.3 等宽胖树

讲者还提到一个变体：等宽胖树，有时也叫折叠 Clos 网络。它的特点是所有节点度固定，实现更简单。

7.4 超立方体

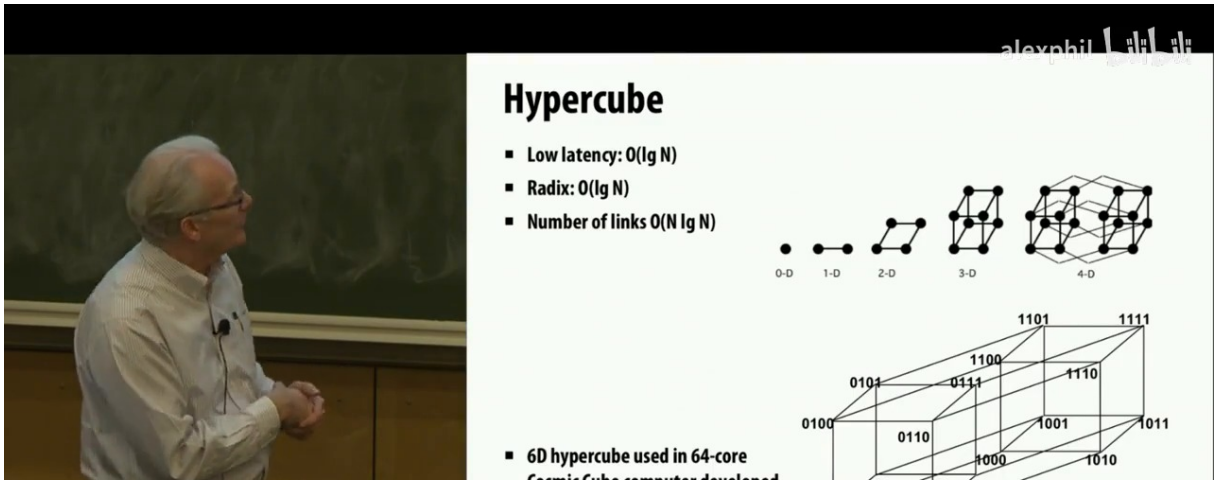


图 24: 超立方体：延迟 $O(\lg N)$、对分带宽随节点数增长，但物理难布线[24]

超立方体的指标很漂亮：延迟 $O(\lg N)$，节点度 $O(\lg N)$，链路数 $O(N \lg N)$，对分带宽随节点数线性增长。

讲者给出 0 维、1 维、2 维、3 维、4 维的递进图示。

但它有一个严重问题：物理上你无法在三维空间里表达更多维度。四维立方体还能靠塞进足够多的导线实现，但规模再往上，导线就成了一团密集的网，无法很好地布线。

于是它有一个漂亮的对分带宽，却有一个糟糕的性质：超过某个限度后，无法嵌入任何可实现的物理结构。

讲者补充说可以推广：不只每个地址位对应一个二选一，而是让每维有更高的度。他说 Cray 造过的一些超算确实用了四维立方体，但不是二元的，每个节点的度更高。

[23] 视频画面时间区间：00:40:00–00:40:15。若为裁剪图，时间区间仍对应原视频画面。

[24] 视频画面时间区间：00:50:00–00:50:15。若为裁剪图，时间区间仍对应原视频画面。

理论指标优秀不等于可实现

超立方体的延迟与对分带宽在所有拓扑中都属最优，但它的连线在三维空间中会交叉成灾。这说明拓扑评估不能只看渐近指标，还要考虑物理布局成本——导线长度、层数、交叉。实际系统因此在指标与可实现性之间折中，选择了胖树或高维环面。

7.5 omega 网络

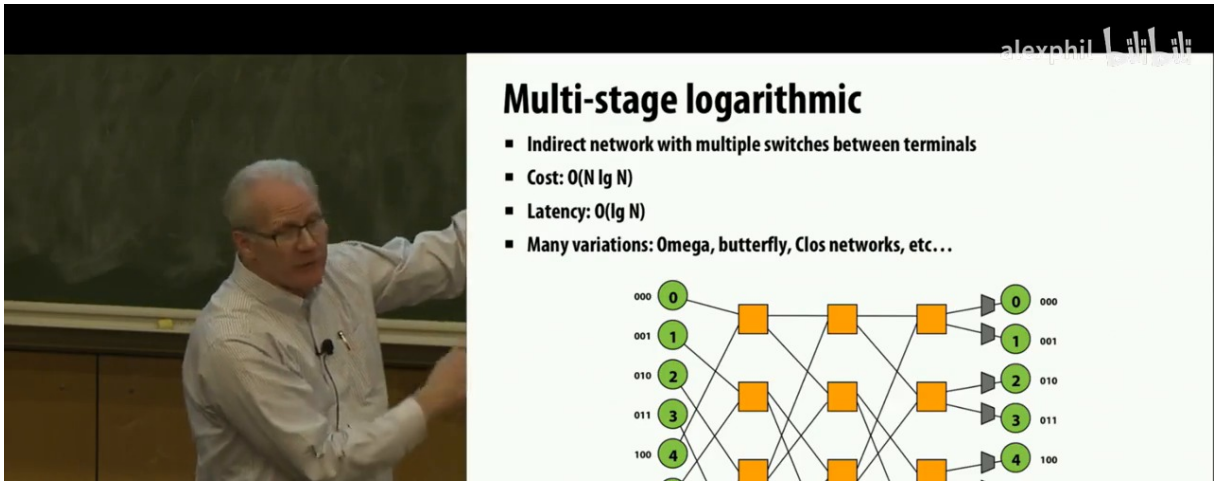


图 25: omega 网络：多级对数网络，用约 $N \log N$ 个开关连接 N 个终端[25]

omega 网络是一个经典设计。讲者指出画法上有误导：左端和右端的节点其实是同一批，只是画成两列便于理解。

实际结构是 N 个节点，中间大约 $N \log N$ 个开关（更准确地说是每级 $N/2$ 个开关、共 $\log N$ 级），提供从一端到另一端的通路。

它的特点是：第一级跨越较大间隔，到末端只做相邻的交换。

$$\text{开关数}_{\text{omega}} = \frac{N}{2} \log_2 N$$

- N：终端节点数；
- $\log_2 N$：级数；
- 每级 $N/2$ 个二选一开关，因此总数与 $N \log N$ 同阶。

7.6 本章小结

树的路由完全由地址位决定，硬件极简但根部是瓶颈。胖树在上层加宽链路，使对分带宽不随切口位置变化，代价是更多连线。等宽胖树让节点度固定以简化实现。超立方体指标最优但物理上难以布线，实际系统用更高节点度的推广版本。omega 网络用 $N \log N$ 个开关构成多级网络。

[25] 视频画面时间区间：00:52:00–00:52:15。若为裁剪图，时间区间仍对应原视频画面。

8 总结与延伸

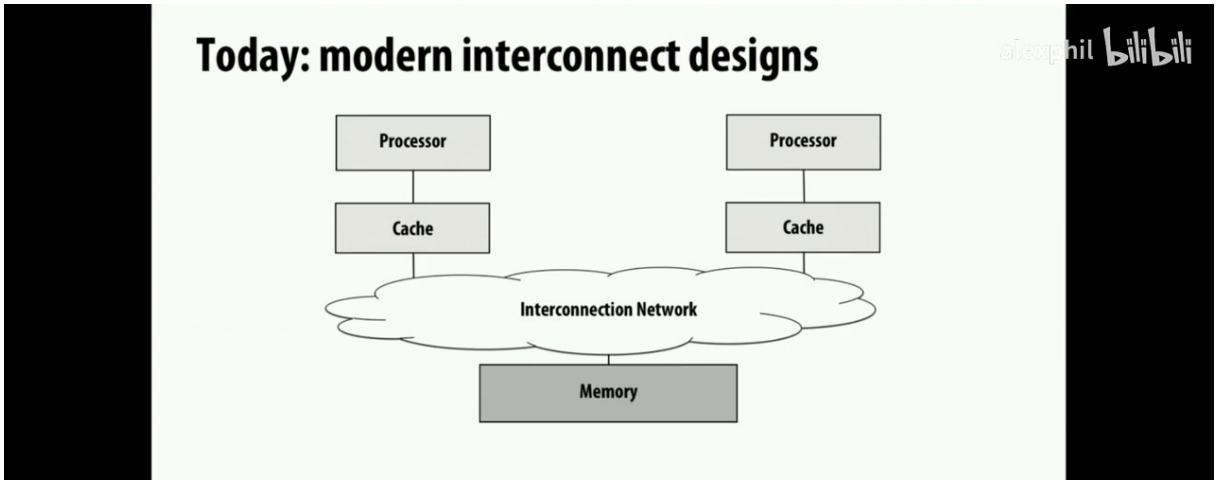


图 26: 现代互连网络设计：处理器、缓存与互连的整体视图[26]

讲者在结尾回到一个现实观察：网络正越来越成为从机器中取得性能的关键问题。他提到一个具体的经验——把相关的作业都集中放在数据中心的同一个角落、以确保它们之间有足够带宽，这件事比直接把网络配置得足够宽、让作业可以随机放置并仍取得不错性能要困难得多。

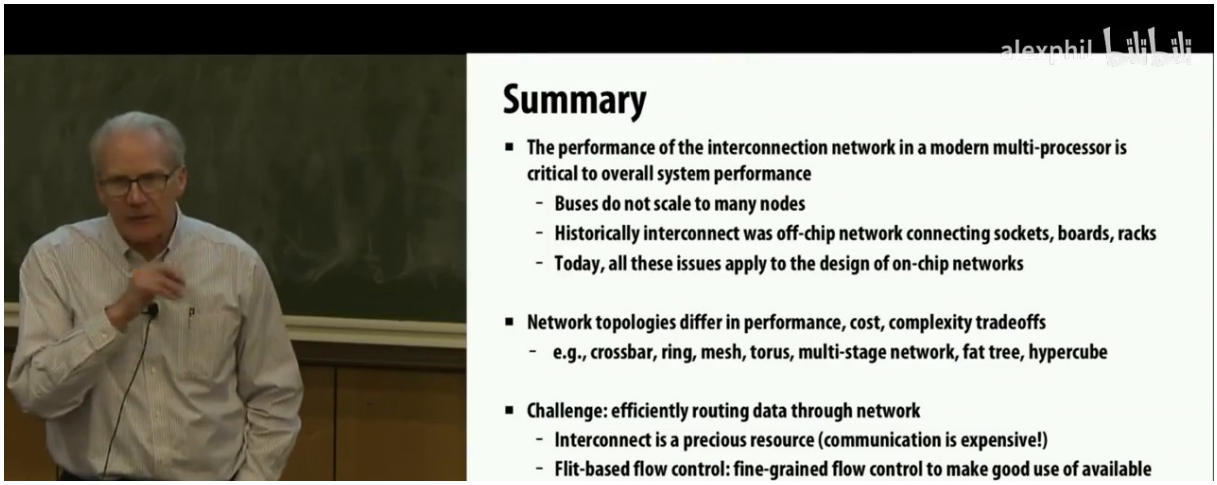


图 27: 总结：互连网络的性能对现代多核处理器的整体性能至关重要[27]

把全讲串起来，这是一份关于”如何把消息送到远处”的设计地图。互连网络连接 HPC 节点，目标与互联网不同：紧密局部、极高性能，可较少考虑故障。最简单形式是总线，请求与响应须物理分离以避免死锁。设计需权衡性能、能耗与可扩展性，厂商希望同一设计覆盖 256 到 1024 等多种规模。四个设计问题是拓扑、路由、缓冲与流量控制；拓扑用直径与对分带宽度量，mesh 为节点数的平方根、环为 1、超立方体为节点数。无局部性负载会让低对分带宽成为瓶颈，而延迟随负载上升是通用规律。具体拓扑从交叉开关（完整图，面积平方增长）到环（$O(N)$ 延迟、恒定对分带宽）、mesh 与环面（呼应局部性）、树与胖树（路由由地址位决定，胖树在上层加宽消除根瓶颈）、超立方体（指标最优但难布线）、omega 网络（$N \log N$ 个开关的多级网络）。

[26] 视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。

[27] 视频画面时间区间：01:12:40–01:12:55。若为裁剪图，时间区间仍对应原视频画面。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 对分带宽是最坏情况指标，不能单独用于判断拓扑优劣。它假设流量均匀跨越中线，而真实应用的通信常有局部性。因此评估时应把对分带宽与应用的访问模式一起看，否则会低估局部性友好的拓扑。
- 拓扑选择由规模与流量模式共同决定，不存在普遍最优。环在四到八核时够用，几十核时 mesh 更好，上百节点则需要胖树。同一套设计要覆盖多个规模，这正是厂商选择层次化方案的原因。
- 物理可实现性是理论指标之外的一等约束。超立方体在所有渐近指标上都优秀，却因布线困难而很少被采用。评价任何拓扑都必须同时问” 能不能造出来”。
- 路由简单性本身有价值。树路由只需地址位运算，无需路由表，因此延迟可预测、硬件成本低。复杂拓扑的灵活路由往往要付出表项与查找延迟的代价。

这堂课的边界在于它是导论：讲者只给出拓扑与指标的比较，未深入具体路由算法、缓冲分配策略与流量控制协议。真实互连的工程细节远超本讲范围。

两个可以直接从本讲数据提出的下一步问题：其一，既然对分带宽是最坏情况指标，那么对于一个通信有明显局部性的应用，应当用哪个指标来替代它做评估？其二，胖树通过在上层增加链路消除了根瓶颈，但上层链路数增加会带来什么新的代价，这个代价在什么规模下开始主导？